

SimpleChat Documentation

CS 550 Advanced OS - Programming Assignment - Part 1

Harun Pekacar - A20607262

Github Link: <https://github.com/0hr/SimpleChat>

Introduction

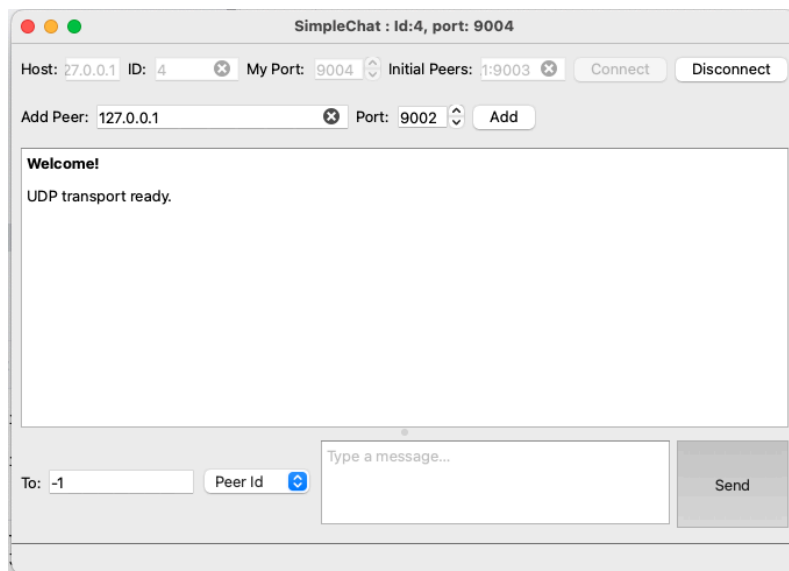
SimpleChat is a peer-to-peer (P2P) messaging application that demonstrates UDP-based, peer-to-peer messaging with anti-entropy. Instances discover neighbors on localhost, exchange vector clocks, and deliver both direct and broadcast chat messages reliably. It's built in C++ and the Qt6 framework. This document provides an overview of the application's design, features, and building, implementation, and basic test cases.

Core Requirements

GUI Implementation

The application provides a basic graphical user interface (GUI) for users to interact with the chat system. The GUI is implemented using the Qt6 framework. GUI has the following requirements;

- Chat log shows received and sent messages with word wrap.
- A multi-line text input field allows users to compose and send messages.
- Destination picker filled with known peer Ids plus the broadcast value '-1'.
- A user can connect/disconnect/Add Peer.



Architecture Overview

UI Layer: SimpleChatMainWindow is the Qt Widgets interface. It has input, displays chat logs, and exposes buttons for connect/disconnect, sending, and adding peers.

Controller Layer: ChatController bridges UI signals to the transport (sendChat, requestPeer) and converts transport callbacks into UI updates.

Transport Layer: Core::UdpPeerTransport implements IChatTransport. It wraps QUdpSocket, manages peer discovery, maintains vector clocks, retransmits unacknowledged packets, and performs anti-entropy.

Network Communication

The chat system uses a UDP-based messaging protocol for communication between peers. Its peers exchange datagrams over UDP

- QUdpSocket is used for network communication, datagram-based data transfer.
- Every packet is a 'QVariantMap' written directly into a datagram via 'QDataStream'. UDP preserves frame boundaries, so no custom length prefix is required. On receipt, the payload is deserialized back into a map; malformed frames are dropped.
- Every datagram is a 'QVariantMap' serialized with 'QDataStream'.
 - **Hello Datagram:** Shares local view so neighbors can expand their peer list.
 - Type (QString): "Hello".
 - Sender (QString): Id of the peer.
 - Origin (QString): Origin Id.
 - Port (quint16): UDP port the sender is listening on.
 - Vector (QVariantMap): vector clock data.
 - Peers (QStringList): known addresses. Used to populate the discovery queue on the receiver.
 - **HelloReply Datagram:** Response to Hello, confirming reachability.
 - Type (QString): "HelloReply".
 - **Summary Datagram:** Anti-entropy digest sent on a timer. Recipients compare vector clocks and request missing chats.
 - Type (QString) : "Summary".
 - Sender / Origin (QString) : Origin/Sender.
 - Destination (QString) : neighbor Id that should consume the summary.
 - Port (quint16) : sender's UDP port.
 - Vector (QVariantMap) : sender's vector clock data.
 - Reply (bool) : false for initial summaries, true when acknowledging one we just received.
 - **Chat Datagram:** User payload. Broadcasts use Destination="-1".
 - Type (QString) : "Chat".
 - Sender (QString) : Sender Id.
 - Origin (QString) : Origin Id.
 - Destination (QString) : target peer Id, or "-1" for broadcast.
 - Recipient (QString) : Recipient Id.

- ChatText (QString) : Chat data.
- Seq (qulonglong) : sequence number.
- Port (quint16) : sender's UDP port.
- **Ack Datagram:** Confirms receipt of a specific chat.
 - Type (QString) : "Ack".
 - Sender (QString) : Sender Id.
 - Destination (QString) : Destination Id.
 - Port (quint16) : sender's UDP port.
 - AckOrigin (QString) : Ack Origin.
 - Seq (qulonglong) : sequence number acknowledged.

Peer Discovery Flow

1. The peer sends Hello to neighbors.
2. The receiver answers with HelloReply.
3. When a previously unknown Id joins peersById, the transport emits peersChanged, which the controller forwards to the UI.

discoverLocalPeers() scans ports around the local bind port in both directions (radius 10 * multiplier 5 * 10 iterations) and pushes candidates onto a throttled queue.

processDiscoveryQueue() drains one entry every 200 ms to avoid bursts of Hello packets.

Chat Flow

1. ChatController::sendChat assigns a new sequence number, records it in the seen set, and delegates to transport->send.
2. The transport caches the message, storeMessage, and either hands it to the destination directly, floods to all peers for broadcasts, or reports an error if the destination Id is unknown.
3. sendChatToPeer annotates the map with Sender, Recipient, Port, writes the datagram, and registers a PendingAck.
4. Receivers call handleChat, persist the payload if it is new, mark the sender as having forwarded it, and emit messageReceived when the destination matches or a broadcast arrives. Broadcasts that are new get forwarded to every other peer that has not acknowledged the message yet.
5. An immediate Ack is routed back to the sender. When handleAck fires, the transport removes the pending entry and updates the neighbor's vector clock.
6. onResendTimeout (every 1.5 s) replays any chats that still lack acknowledgments, up to five attempts before reporting an error.
7. onAntiEntropyTimeout (every 2 s) pushes a Summary to each peer. If a neighbor advertises a lower sequence number for an origin, we replay the missing chats from historyByKey. Replies prevent the endless ping-pong of summaries.

The UDP P2p Network

In a p2p network, each chat application links to neighboring peers.

Establishment of UDP P2P Network

In SimpleChat, the ring is established at startup based on command-line arguments:

- **--port:** Specifies the port on which the current instance will listen for incoming connections.
- **--peers:** Comma-separated list of other peers.
- **--test_message:** Specifies the message to send to the peer.
- **--test_peer:** Specifies the message to send to the peer.
- **--test_wait_time:** Specifies the delay before the test message is sent (in milliseconds).
- **--test_count:** Specifies how many times to send the test message.

Each instance acts as both a server and a client. Also, the udp p2p network can be established on the GUI.

Advantages

- There is no central server. So the network is decentralized.
- Minimal protocol overhead keeps latency low and makes broadcasts cheap. It is perfect for forwarding the same chat to many peers.
- Stateless datagrams avoid head-of-line blocking. It means if one packet is lost, others still flow, while retries happen in parallel.

Disadvantages

- The package can be lost. Acknowledgments, resend timers, and vector clocks add significant complexity.
- Datagrams can arrive out of order or duplicated.

Build and Run the Project

A helper script, `build_run.sh`, is provided to build and run 4 nodes. They will be connected in a udp p2p with IDs 1, 2, 3, and 4 on ports 9001, 9002, 9003, and 9004.

1. **Make the script executable:** `chmod +x build_run.sh`
2. **Run the script:** `./build_run.sh`

Build Manually

1. **Create build directory:** mkdir build
2. **Go into "build" directory:** cd build
3. **Generate build files:** cmake ..
4. **Compile the project:** cmake --build .

Run the project

To run an instance of the application, you need to specify the user ID, the port, and the peers.

```
./build/SimpleChat  
--id <id>  
--port <port>  
--peers <comma-separated-list-of-other-peer-ids>
```

For example, to start a chat client with ID "1" on port 9001, and there are 9002, 9003, 9004 peers, you would run:

```
./build/SimpleChat --id 1 --port 9001 --next 9002 --peers  
127.0.0.1:9002,127.0.0.1:9003,127.0.0.1:9004
```

To create a ring of 4 peers, you would run the following commands in separate terminals:

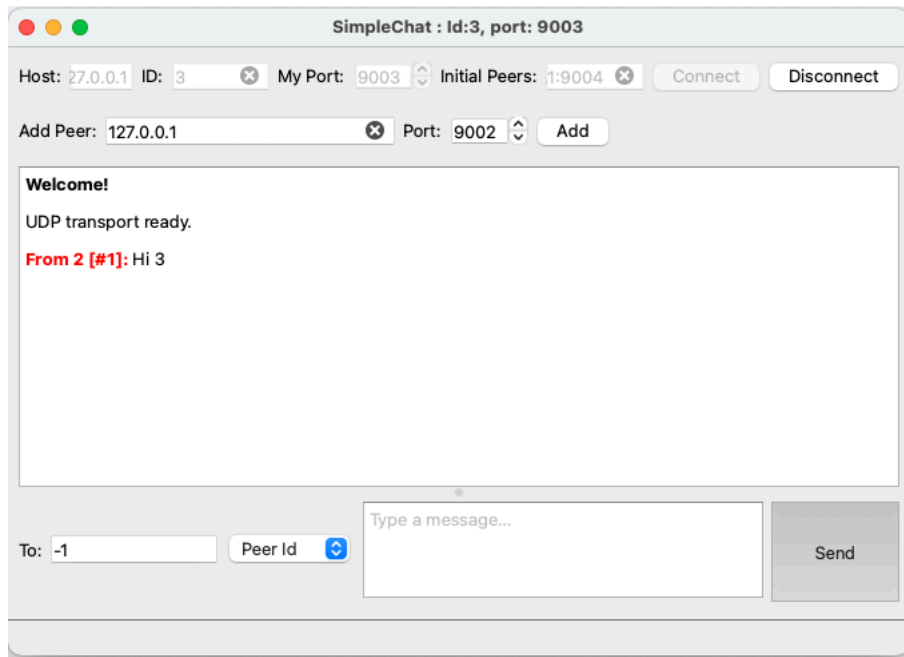
```
./build/SimpleChat --id 1 --port 9001 --peers 127.0.0.1:9002,127.0.0.1:9003,127.0.0.1:9004  
./build/SimpleChat --id 2 --port 9002 --peers 127.0.0.1:9001,127.0.0.1:9003,127.0.0.1:9004  
./build/SimpleChat --id 3 --port 9003 --peers 127.0.0.1:9001,127.0.0.1:9002,127.0.0.1:9004  
./build/SimpleChat --id 4 --port 9004 --peers 127.0.0.1:9001,127.0.0.1:9002,127.0.0.1:9003
```

Basic Test Cases

Test Case 1

Test Case: Peer 2 sends "Hi 3 " to Peer 3

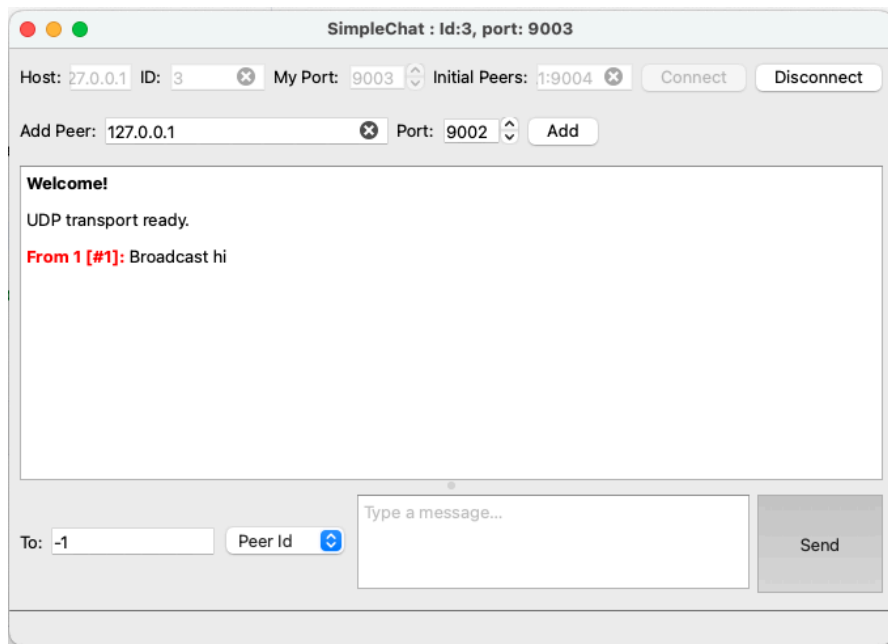
Expect: Message Shows on Peer 3 only.



Test Case 2

Test Case: Peer 1 sends "Broadcast hi"

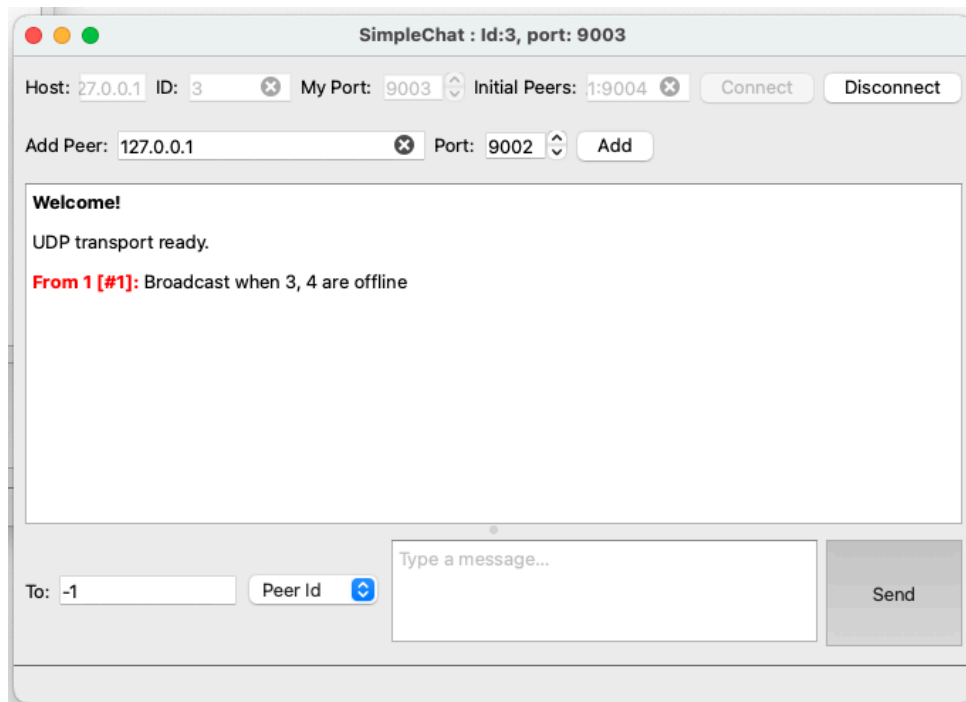
Expect: Message Shows on Peer 2, 3, 4



Test Case 3

Test Case: Peer 1 sends "Broadcast hi" when 3, 4 are offline

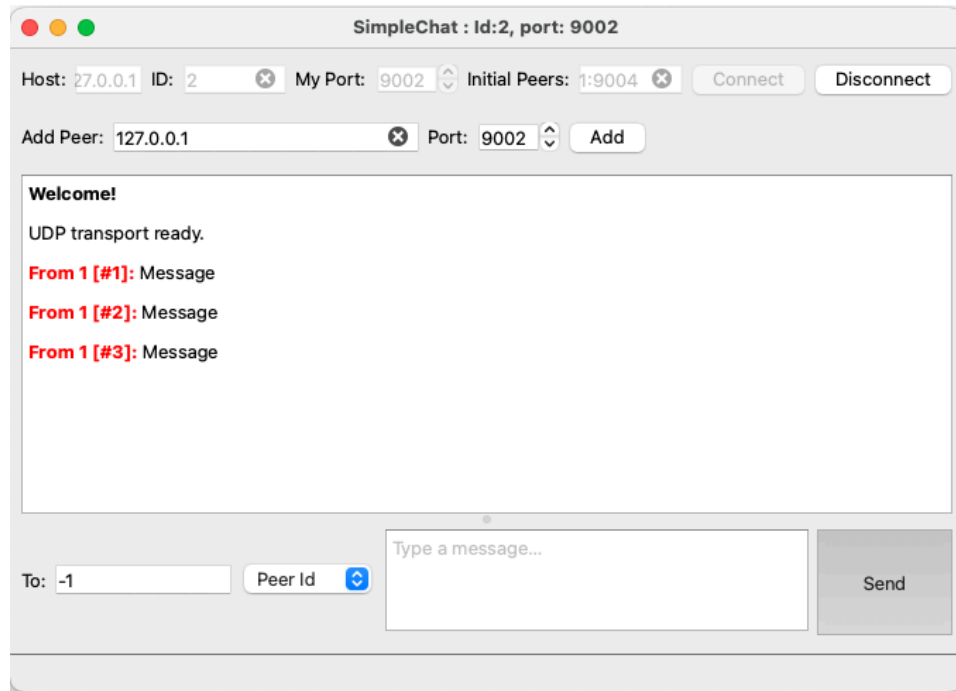
Expect: Message Shows on Peer 2, and when 3, 4 join, they get broadcast messages



Test Case 4

Test Case: Peer 1 sends messages to Peer 2 quickly m#1, m#2, m#3

Expect: On Peer 2, messages appear m#1, m#2, m#3 in order from Peer 1



Test Case 5

Test case: Peer 1 quickly sends to Peer 4: m#1p1, m#2p1, m#3p1; then Peer 3 sends to Peer 4: m#1p3, m#2p3, m#3p3; finally, Peer 1 sends to Peer 4: m#4p1.

Expect: On Peer 4, messages appear m#1p1(sequence: 1), m#2p1(sequence: 2), m#3p1(sequence: 3), m#1p3(sequence: 1), m#2p3(sequence: 2), m#3p3(sequence: 3), m#4p1((sequence: 4) in order.

SimpleChat : Id:3, port: 9003

Host: 127.0.0.1

ID: 3

My Port: 9003

Initial Peers: 1,127.0.0.1:9004

Connect

Disconnect

Add Peer: 127.0.0.1

Port: 9002

Add

Welcome!

UDP transport ready.

From 1 [#1]: Message FROM Peer 1

From 1 [#2]: Message FROM Peer 1

From 1 [#3]: Message FROM Peer 1

From 2 [#1]: Message FROM Peer 2

From 2 [#2]: Message FROM Peer 2

From 2 [#3]: Message FROM Peer 2

From 4 [#1]: Message FROM Peer 4

From 4 [#2]: Message FROM Peer 4

From 4 [#3]: Message FROM Peer 4

To: -1

Peer Id

Type a message...

Send